

MODULE - II

Introduction to Hadoop: Introducing Hadoop, Why Hadoop, Why not RDBMS, RDBMS VS Hadoop, History of Hadoop, Hadoop Overview, use case of Hadoop, HDFS, Processing data with Hadoop, Managing resources & applications with Hadoop YARN.

Introduction to Map Reduce Programming: Introduction, Mapper, Reducer, Combiner, Partitioner, Searching, Sorting, Compression.

5.1 Introducing Hadoop

- Today, Big Data seems to be the buzz word!
- The world over are beginning to realize that there is huge volume of untrapped info.
- This varied variety of data is spread across the n/w.
- Let's see some of the examples to get an idea of the amount of data which gets generated every day.

1) Everyday:

- a) New York Stock Exchange generates 1.5 billion shares & trade data
- b) Facebook stores 2.7 billion comments & likes
- c) Facebook processes about 24 petabytes

2) Every minute:

- a) Facebook users share nearly 2.5 million pieces of content
- b) Twitter users tweet nearly 300,000 times
- c) Instagram users post nearly 220,000 new photos
- d) YouTube users upload 72 hours of new video content
- e) Apple users download nearly 50,000 apps
- f) Google receives over 4 million search queries

3) Every second:

- a) Banking apps process more than 10,000 credit card transactions.

Data: The treasure Trove.

- 1) provides business advantages such as generating product recommendations, inventing new products, analyzing the market, & many more.
- 2) provides few early key indicators that can turn the fortune of business.
- 3) provides room for precise analysis. If we have more data for analysis, then we have greater precision of analysis.

To process, analyze, & make sense of these diff data, we need a system that scales & addresses the challenges.



"I am inundated with data"
How to store terabytes of mounting data?

"I have data in varied sources.
I have data that is rich in Variety - structured, semi-structured & un-structured." How to work with data that is so very different?



"I need this data to be processed quickly.
My decision is pending." How to access the information quickly?



Fig: Challenges with big volume, Variety, & Velocity of data.

5.2 Why Hadoop?

(3)

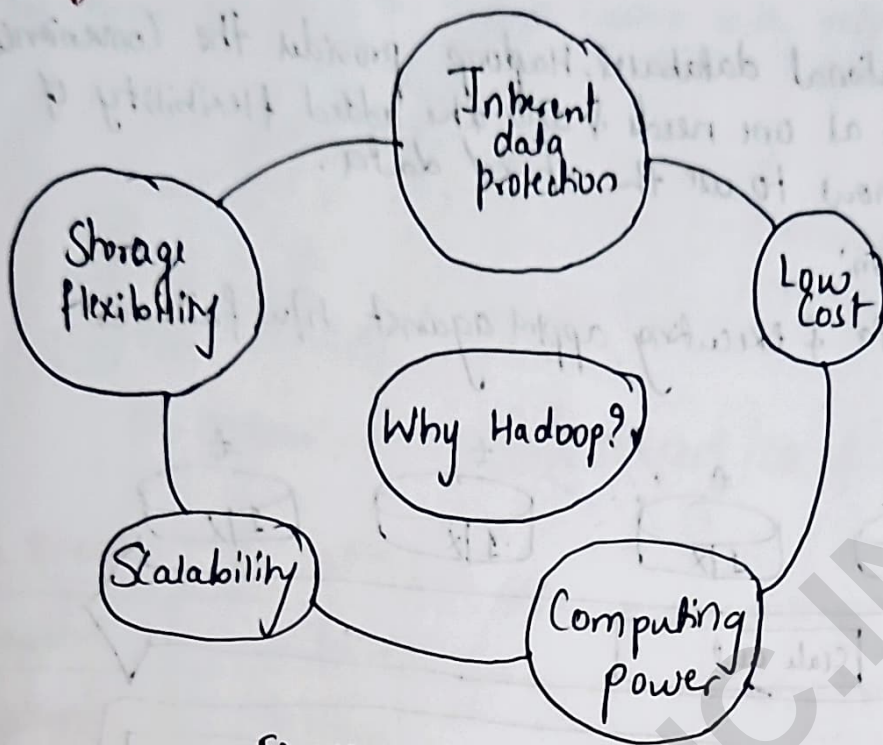


Fig: Key Considerations of Hadoop

The key consideration of Hadoop is

Its capability to handle massive amounts of data, different categories of data - fairly quickly.

The other considerations are:

1. Low cost:

Hadoop is open-source & uses commodity h/w to store enormous quantities of data.

2. Computing power:

→ ∵ It uses distributed computing model which processes large volume of data quickly

→ The more the no. of computing nodes, the more the processing power at hand.

3. Scalability:

This boils down to simply adding nodes as the system grows & requires much less administration.

4. Storage flexibility:

→ Unlike traditional relational databases, Hadoop provides the convenience of storing as much data as one needs & also the added flexibility of deciding later as to how to use the stored data.

5. Inherent data protection:

→ Hadoop protects data & executing apps against h/w. failure.

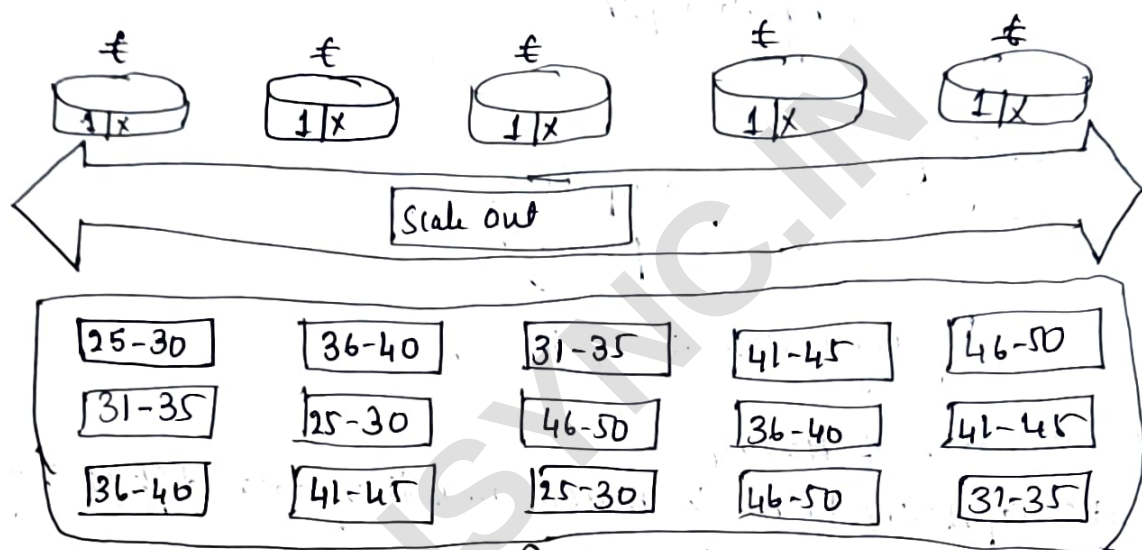


Fig: Hadoop framework (distributed file system, commodity h/w)

→ Hadoop uses commodity h/w, distributed file system, & distributed computing as shown in above fig

→ The group of machines are gathered together; it is known as a cluster.

→ With this new paradigm, data can be managed with hadoop as

- i) Distributed the data & duplicate chunks of each data files across several nodes
- ii) Locally available compute resource is used to process each chunk of data in place
- iii) Hadoop framework handles failover smartly & automatically.

5.3 Why NOT RDBMS?

→ It is not suitable for storing & processing large files, images & videos.

→ It is not good choice when it comes to advanced analytics involving

ML
 → The below fig shows the RDBMS System with respect to cost & storage. (5)



Fig: RDBMS with respect to cost / GB of storage.

5.4 RDBMS VS Hadoop:

Parameters	RDBMS	Hadoop
System	RDBMS	Node Based Flat structure.
Data	Suitable for structured data	Suitable for structured, unstructured. Supports variety of data formats in real time such as XML, JSON, text based, flat file formats etc.
Processing	OLTP	Analytical, Big Data processing
Choice	When data needs consistent relationship	Big data processing, which does not require any consistent relationship b/w data.
Processor	Needs expensive h/w or high-end processors to store huge volumes of data.	In a hadoop cluster, a node requires only a processor, a n/w card, & few hard drives.
Cost	Cost around \$10,000 to \$14,000 per terabytes of storage	Cost around \$4,000 per terabytes of storage.

5.6 History of Hadoop:

- It was created by Doug Cutting
- It is a part of the Apache Nutch (Yahoo) project & also a part of the Lucene project.

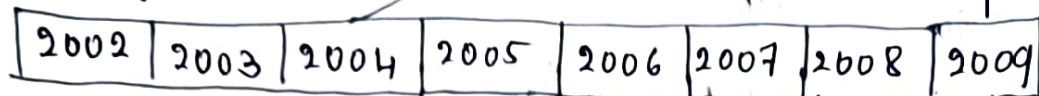
The Name 'Hadoop'

6

- It is not an acronym, it's a made-up name.
 - The name my kid gave a stuffed yellow elephant
- Dough added DFS & Map Reduce to Nutch

Dough Cutting & Mike Cafarella,
started working on Nutch

Cloudera founded



Google published
GFS & Map Reduce
Google File system

Yahoo hired Dough.
Hadoop spins out Nutch

Dough Cutting
joined Cloudera

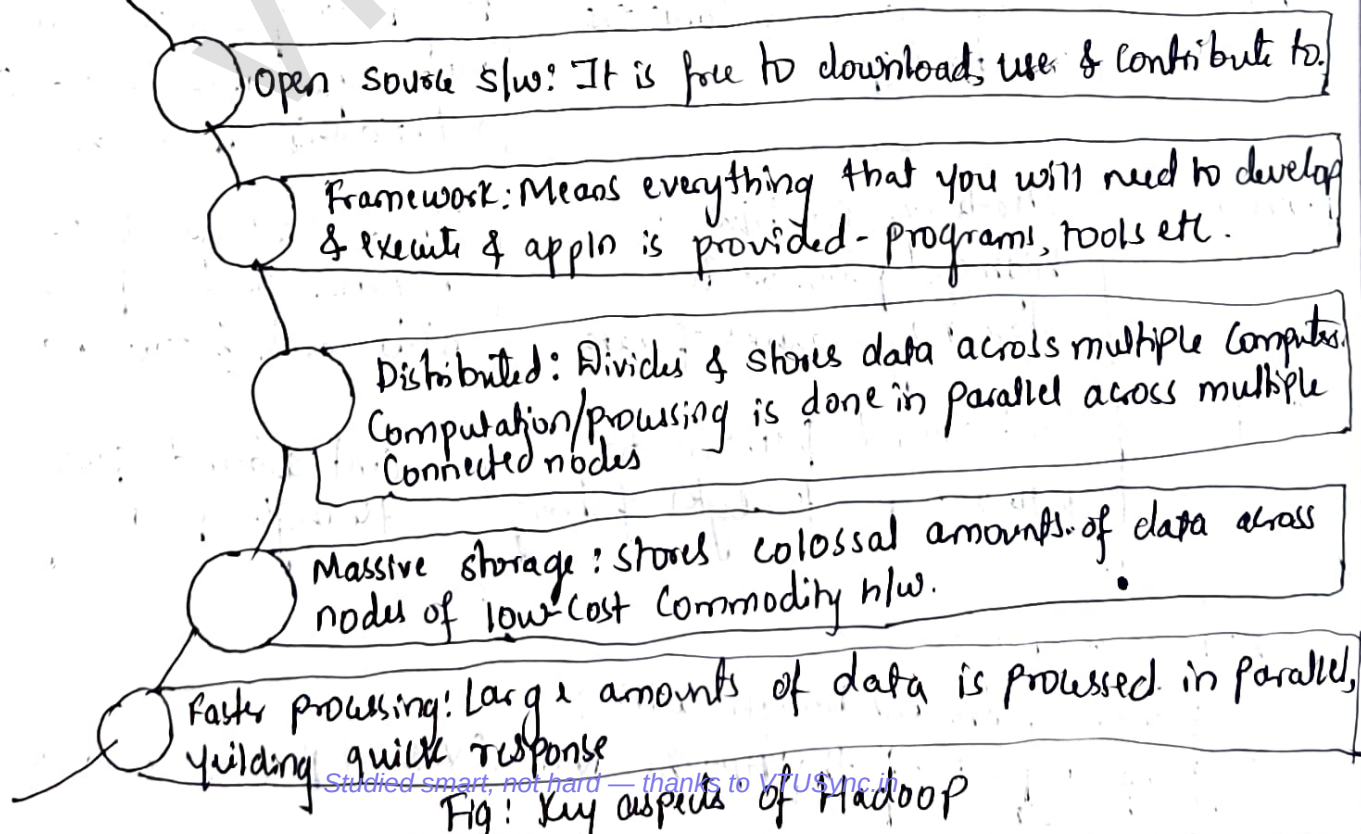
Fig: Hadoop History.

5.7 Hadoop Overview:

Hadoop accomplishes two tasks

1. Massive data storage
2. Faster data processing.

* Key Aspects of Hadoop



* Hadoop Components

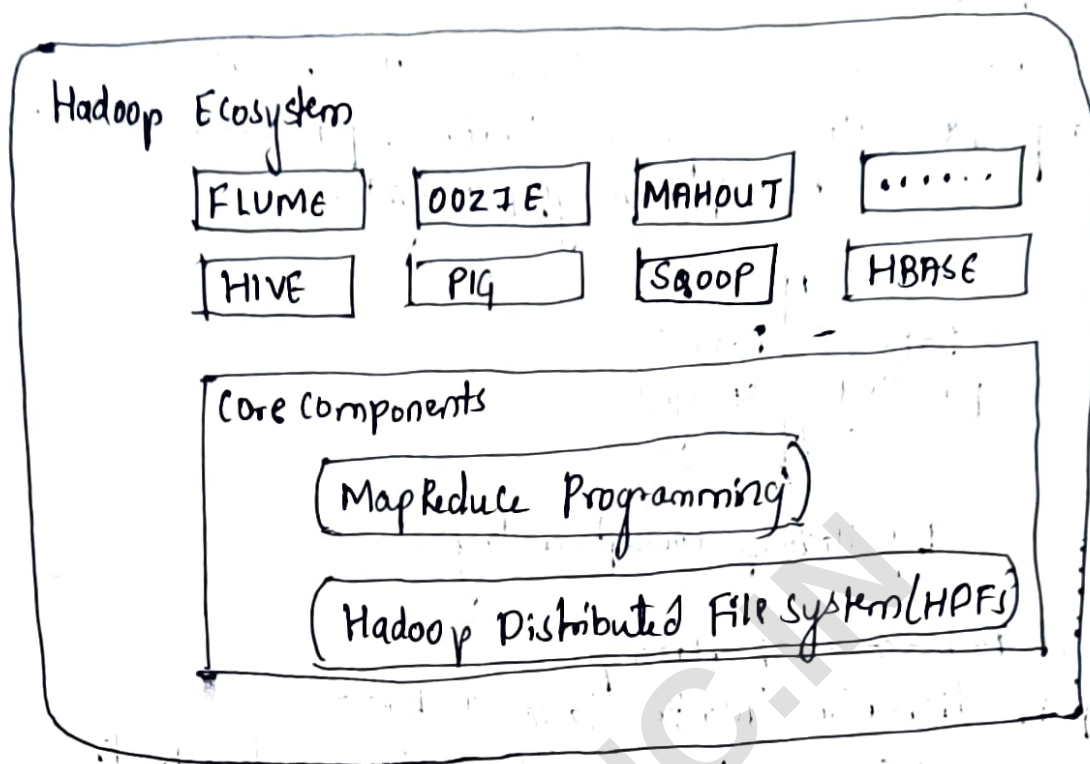


Fig: Hadoop Components.

Hadoop Core Components

1) HDFS

a) Storage Component.

b) Distributes data across several nodes

c) Natively redundant.

2) MapReduce:

a) Computational framework.

b) Splits a task across multiple nodes

c) Processes data in parallel.

Hadoop Ecosystem:

→ It supports projects to enhance the functionality of Hadoop Core Components. The Ecosystem projects are:

i. Hive

ii. PIG

iii. SQOOP

iv. HBase

v. Flume

vi. Oozie

vii. Mahout

★ Hadoop Conceptual layer

→ It is divided into

- i) Data storage layer — stores huge volume of data
- ii) Data processing layer — processes data in parallel to extract richer & meaningful insight.

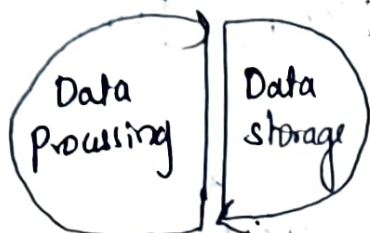


Fig: Hadoop Conceptual layer.

★ High-level Architecture of Hadoop

→ Hadoop is distributed Master-Slave architecture.

→ Master node is known as Namenode & slave nodes are known as Datanodes

→ Below fig depicts Master-slave architecture of hadoop framework

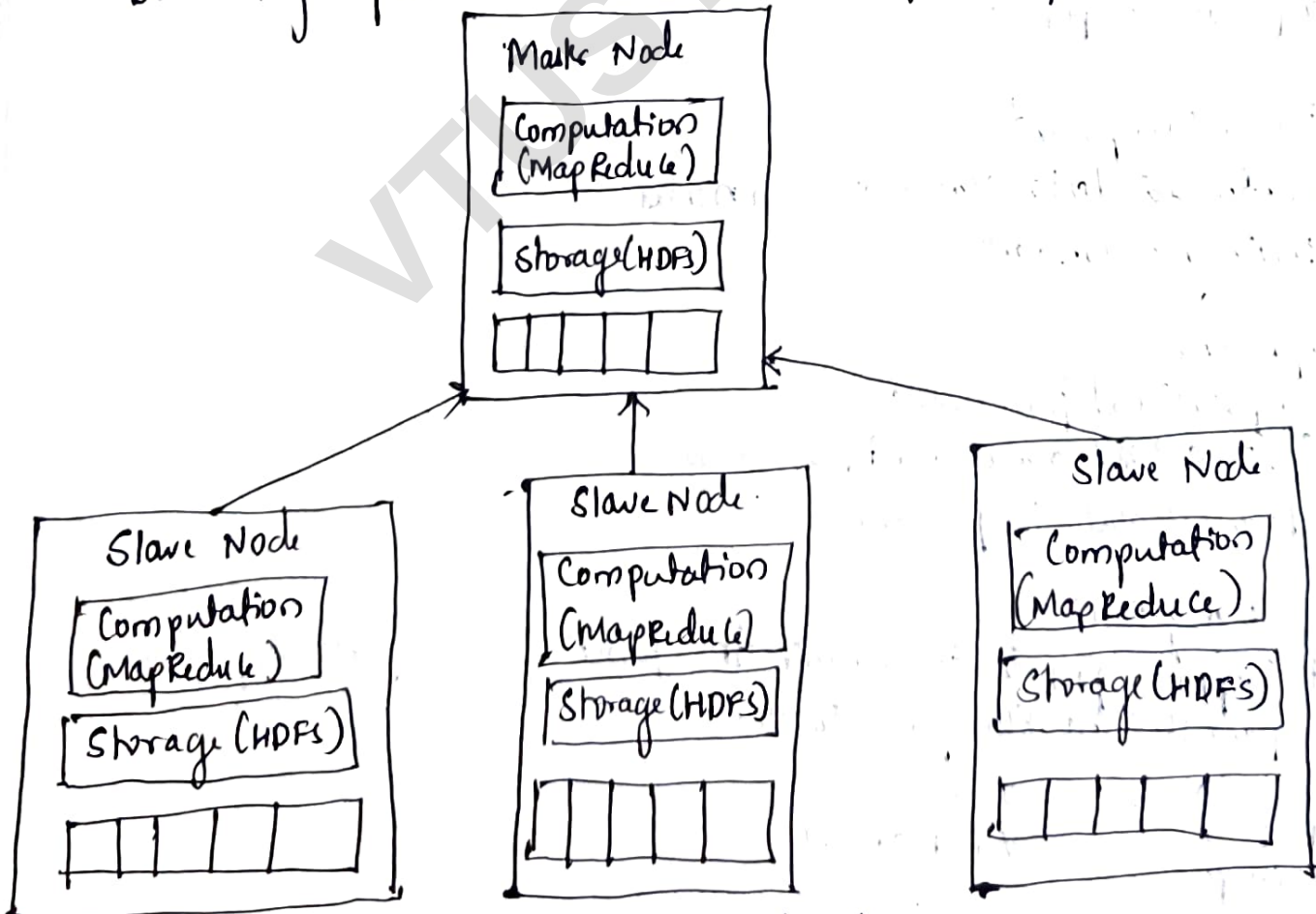


Fig: Hadoop high-level architecture.

Key Components of Master Node

1) Master HDFS:

- Main responsibility is partitioning data storage across slave nodes
- It also keep track of locations of data on datanodes

2) Master MapReduce: It decides & schedules computation task on slave nodes

5.8 USE CASE OF HADOOP:

Clickstream Data

- Clickstream data (mouse clicks) helps you to understand the purchasing behavior of customers.
- Clickstream helps online marketers to optimize their product web pages, promotional content etc. to improve their business

Clickstream Data Analysis using Hadoop - Key Benefits		
Joins clickstream data with CRM & sales data	Stores years of data without much incremental cost	Hive or pig script to analyze data

Fig: clickstream data analysis

→ It provides 3 key benefits

- 1) Hadoop helps to join clickstream data with other data sources such as Customer Relationship Management Data
 - The additional data provides the much needed info to understand customer behavior
- 2) Hadoop's scalability property helps you to store years of data without incremental cost.
- 3) Business analysts can use Apache pig or Apache Hive for website analysis.

5.10 HDFS (Hadoop Distributed File System)

★ HDFS Daemons

1) Name Node.

- HDFS breaks large file into smaller pieces called blocks.
- NameNode uses a rackID to identify Datanodes
- A rack is a collection of Datanodes within the cluster
- NameNode keeps backs of block of a file which is placed on various Datanodes
- NameNode manages file-related operations such as read, write, create & delete. The main job is managing the File System NameSpace.
- NameSpace is a collection of files in the cluster.
- ~~NameSpace~~ NameNode stores HDFS namespace.
- File system namespace includes mapping of blocks to file, file properties & is stored in a file called FsImage.
- ~~When NameNode starts up, it reads FsImage & Editlog from disk & applies~~
- NameNode uses an EditLog (transaction log) to record every transaction happens to the file system.
- When nameNode starts up, it reads FsImage & Editlog from disk & applies all transactions from Editlog to in-memory representation of FsImage.
- It flushes out new version of FsImage on disk & truncates the old Editlog bcz the changes are updated in the FsImage.

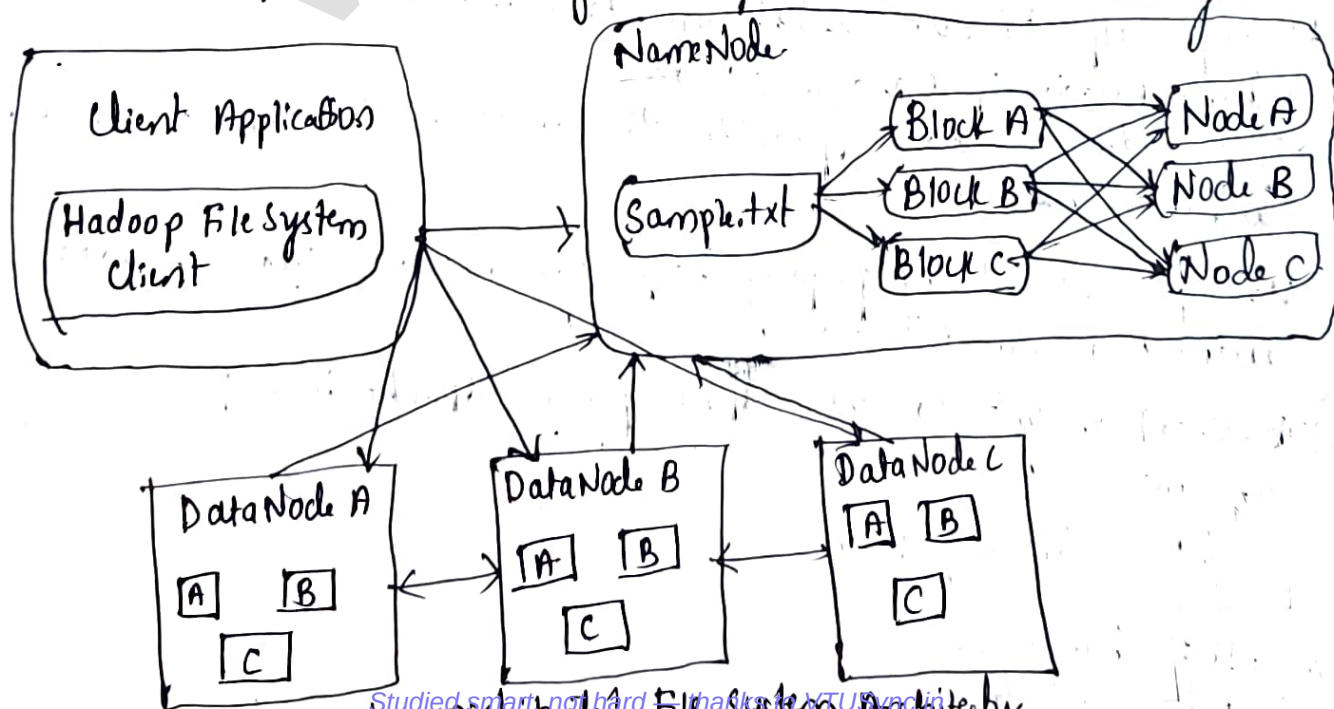


Fig: Hadoop Distributed File System Architecture

NameNode - Manages File Related Operations	
FSImage - File, in which entire file system is stored	EditLog - Records every transaction that occurs to file system metadata

Fig: NameNode.

2) DataNode.

- There are multiple DataNodes per cluster.
- During pipeline read & write DataNodes communicate with each other.
- A DataNode also continuously sends "heartbeat" message to NameNode to ensure the connectivity b/w NameNode & DataNode.
- If there is no heartbeat from DataNode, the NameNode replicates that DataNode, within the cluster & keeps on running as if nothing had happened.

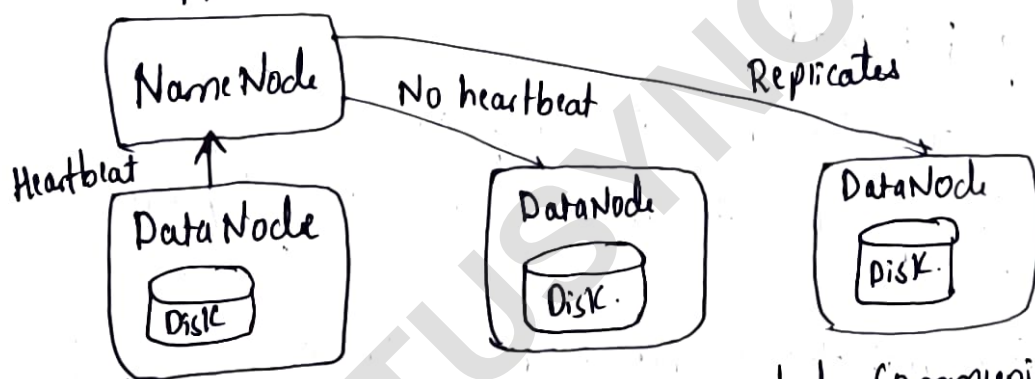


Fig: NameNode & DataNode Communication.

3) Secondary NameNode

- It takes a snapshot of HDFS metadata at intervals specified in the Hadoop Configuration.
- The memory requirements of Secondary NameNode are the same as NameNode, it is better to run NameNode & Secondary NameNode on different machines.
- If failure of the NameNode, the secondary NameNode can be configured manually to bring up the cluster.

* ANATOMY OF FILE READ

The below Fig describes the anatomy of File Read.

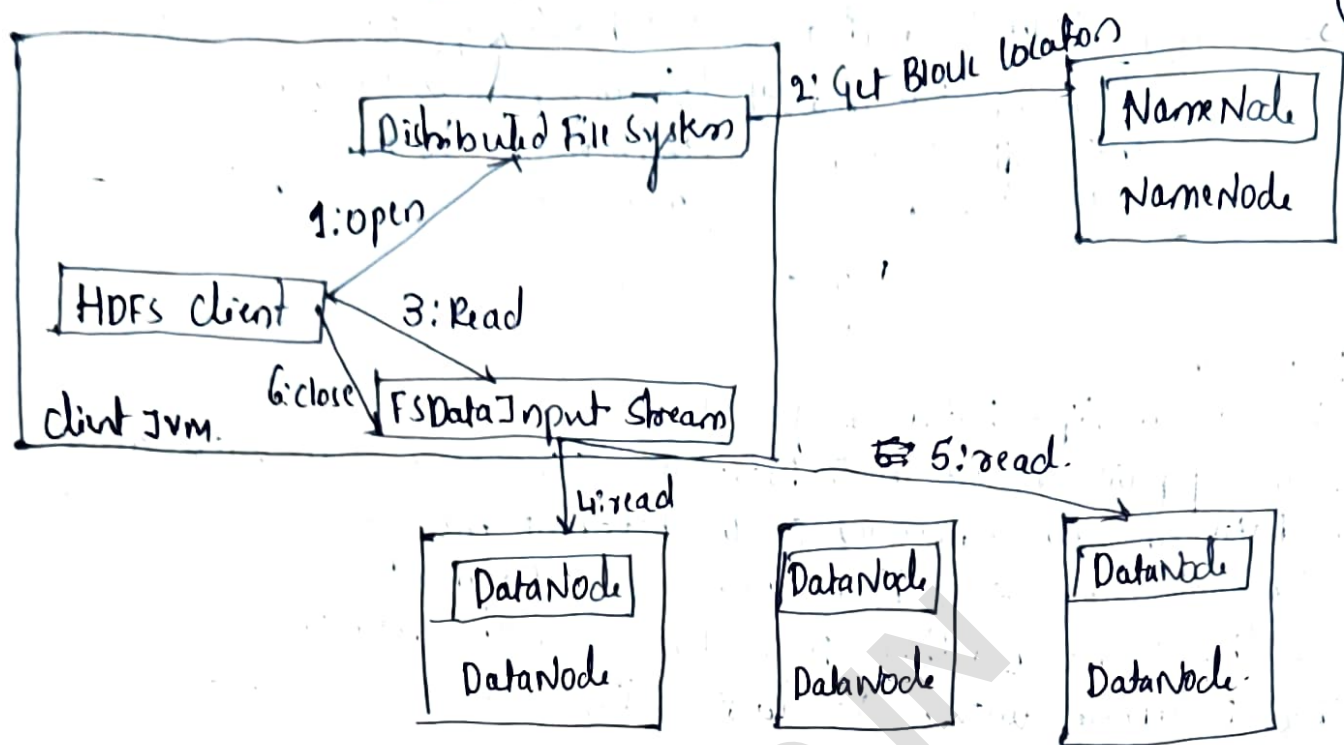


Fig: File Read.

The steps involved in the File Read are as follows:

- 1) The client opens the file that it wishes to read from by calling `open()` on the DFS.
- 2) DFS communicates with the NameNode to get location of data blocks.
 - NameNode returns with the address of the DataNodes that the datablocks are stored on.
 - Subsequently, the DFS returns an `FSDataInputStream` to client to read from the file.
- 3) client then calls `read()` on the stream `DFSInput Stream`, which has addresses of DataNodes for the first few blocks of the file.
- 4) client calls `read()` repeatedly to stream the data from the DataNode.
- 5) when end of the block is reached, `DFSInputStream` closes the connection with the DataNode.
 - It repeats the steps to find the best DataNode for the next block & subsequent blocks.
- 6) when the client completes the reading of the file, it calls `close()` on

the FSDataInputStream to close the connection

* ANATOMY OF FILE WRITE

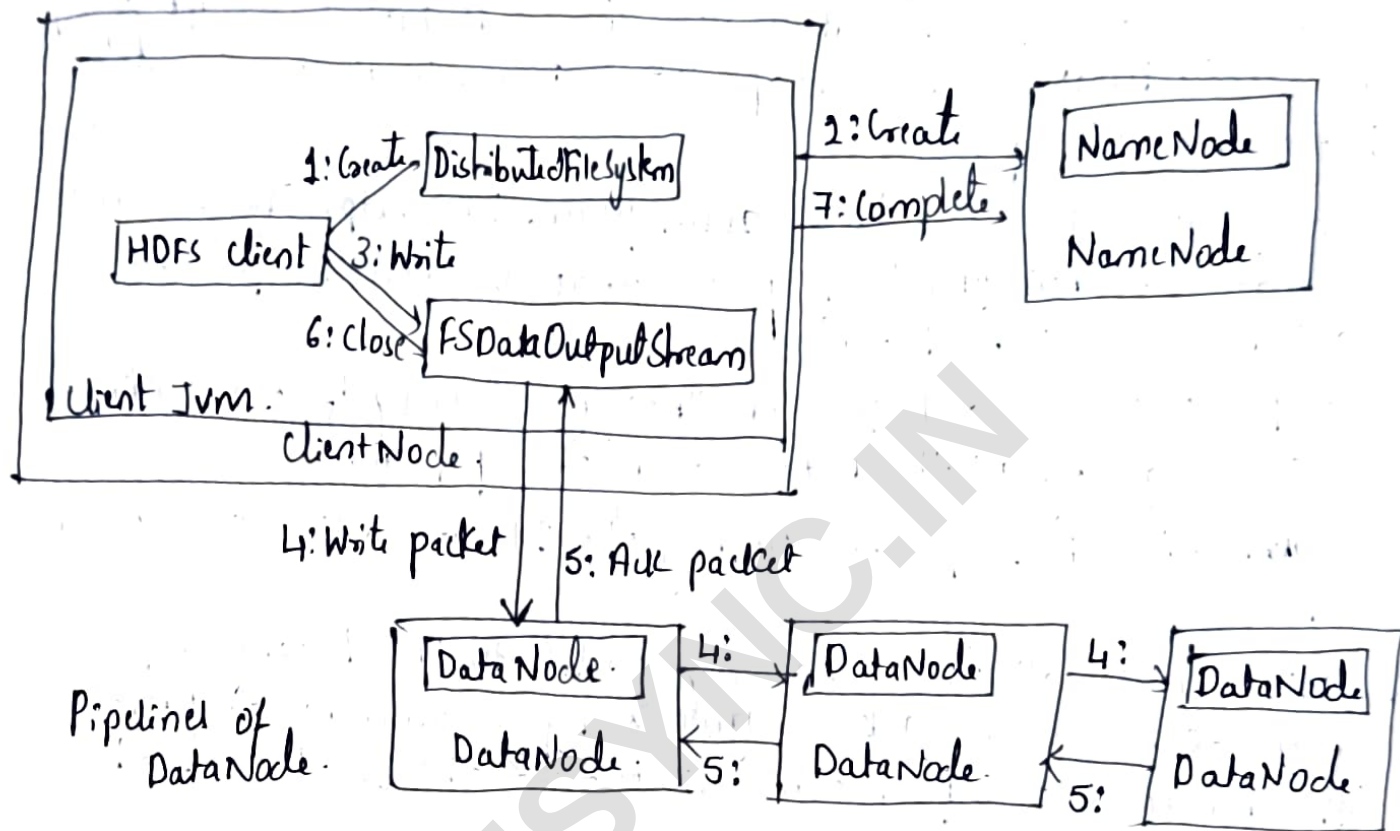


Fig: File Write.

The steps involved in anatomy of File Write are as follows:

1. The client calls `Create()` on DFS to create a file.
2. An RPC call to the NameNode happens through the DFS to create a new file.
 - The NameNode performs various checks to create a new file.
 - The DFS returns an `FSDataOutputStream` to the client to perform write.
3. As the client writes data, data is split into packets by `DFSOutputStream`, which is then written to an internal queue, called data queue.
 - `DataStreamer` consumes the data queue.
 - The `DataStreamer` requests the NameNode to allocate new blocks by selecting a list of suitable DataNodes to store replicas.

- This list of DataNodes make a pipeline. Here, we will go with the default replication factor of three, so there will be three nodes in the pipeline for the first block.
- 4. DataStreamer streams the packets to the first DataNode in the pipeline.
 - It stores packet & forwards it to the second DataNode in the pipeline.
 - The second DataNode stores the packet & forwards it to the third DataNode in the pipeline.
- 5. In addition to the internal ~~queue~~ queue, DFS OutputStream also manages an "Ack queue" of packets that are waiting for the acknowledgement by DataNodes.
 - A packet is removed from the "Ack queue" only if it is ~~acknowledged~~ acknowledged by all the DataNodes in the pipeline.
- 6. When the client finishes writing the file, it calls close() on the stream.
- 7. This flushes all the remaining packets to the DataNode pipeline & waits for relevant acknowledgments before communicating with the NameNode to inform the client that the creation of the file is complete.

* Replica Placement Strategy

Hadoop Default Replica Placement Strategy

- As per the Hadoop Replica Placement Strategy, first replica is placed on the same node as the client.
- Then it places second replica on a node that is present on different rack.
- It places the third replica on the same rack as second, but on a different node in the rack.
- Once replica locations have been set, a pipeline is built.



Fig: Replica Placement Strategy

Working with HDFS Commands:

- 1) To get the list of directories & files at the root of HDFS
`hadoop fs -ls /`
- 2) To get the list of complete directories & files of HDFS.
`hadoop fs -ls -R /`
- 3) To create a directory in HDFS
`hadoop fs -mkdir /sample`
- 4) To copy a file from local file system to HDFS.
`hadoop fs -put /root/sample/test.txt /sample/test.txt`
- 5) To copy a file from HDFS to local file system.
`hadoop fs -get /sample/test.txt /root/sample/testsample.txt`
- 6) To copy a file from local file system to HDFS via CopyFrom local Command.
`hadoop fs -copyFromLocal /root/sample/test.txt /sample/testsample.txt`
- 7) To copy a file from Hadoop file system to local file system via CopyTo local Command.

`hadoop fs -copyToLocal /sample/test.txt /root/sample/testsample1.txt`

8) To display the contents of an HDFS file on console.

`hadoop fs -cat /sample/test.txt`

9) To copy a file from one directory to another on HDFS.

`hadoop fs -cp /sample/test.txt /sample1`

10) To remove a directory from HDFS

`hadoop fs -rm -r /sample1.`

~~Special Features of HDFS~~

~~1. Data Replication:~~